

자료구조및실습: 2018년 1학기

제 12 주:

정렬 성능 비교



강 지 훈

jhkang@cnu.ac.kr

충남대학교 컴퓨터공학과



© J.-H. Kang, CNU

[제 12 주]

정렬 알고리즘들의 성능 비교

□ 실습 목표

■ 이론적 관점

- 삽입 정렬과 퀵 정렬 알고리즘 각각의 개념과 특성 이해
- 데이터의 특성이 정렬의 성능에 미치는 영향의 이해

■ 구현적 관점

- 정렬 알고리즘의 구현
- 성능 측정 방법
- 실험 데이터 생성 방법
- 실험 관리 방법

과제에서 해결할 문제



□ [프로그램 12] 성능 측정

- 삽입 정렬과 퀵 정렬 알고리즘을 각각 구현하여, 그 성능을 측정하고, 비교 분석한다.
- 데이터의 특성에 따른 측정
 - 오름차순 (ascending), 내림차순 (descending), 무작위 (Random) 데이터 각각에 대하여, 각 정렬 알고리즘의 성능을 측정한다.
- 데이터의 크기 별 측정
 - 하나의 측정에 대하여, 데이터의 크기를 달리하여 측정한다.
- 비교 분석
 - 이론적으로 분석하여 얻은 Big-Oh 에 비추어 볼 때, 크기가 증가함에 따라 얻어진 성능 측정 값이 Big-Oh 의 예상과 어느 정도 일치하는지 확인한다.
 - 예:
 - ◆ 삽입 정렬을 사용하여 무작위 데이터를 정렬할 경우 시간 복잡도는 $O(n^2)$ 이다.
 - ◆ 데이터의 크기가 3배 증가하면, 성능 측정 값은 9 배 증가한 값을 얻을 것으로 예상이 된다. 자신의 결과가 정말 그렇게 나왔는지 확인하는 것이다.

□ 입출력

■ 입력:

- 없음
- 필요한 데이터는 프로그램에서 생성

■ 출력: 성능 측정 결과:

- 데이터 크기 변화에 따른 성능 측정 결과
 - ◆ 1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000

이 과제에서 필요한 객체는?

- Class "AppController"

- Class "AppView"

- Model

- Class "DataGenerator"

- Class "ExperimentManager"

- Class "Experiment"

- Class "ParameterSet"

- Abstract Class "Sort<E>"

- Class "InsertionSort<E>" extends "Sort<E>"

- Class "QuickSort<E>" extends "Sort<E>"

- Enum "ListOrder"

- Class "Timer"

출력의 예

<<< 정렬 성능 비교 프로그램을 시작합니다 >>>

>> 2 가지 정렬의 성능 비교: 삽입, 퀵 <<

> 오름차순 데이터를 사용하여 실행한 측정:

	<Insertion Sort>	<Quick Sort>
[1000]	5	557
[2000]	10	2185
[3000]	13	4866
[4000]	19	8646
[5000]	27	17656
[6000]	33	19246
[7000]	35	26994
[8000]	42	37132
[9000]	42	51646
[10000]	49	57863

> 내림차순 데이터를 사용하여 실행한 측정:

	<Insertion Sort>	<Quick Sort>
[1000]	968	593
[2000]	3654	2431
[3000]	11365	6779
[4000]	16745	11381
[5000]	26705	18949
[6000]	37453	26925
[7000]	49529	32725
[8000]	66185	42986
[9000]	77545	56056
[10000]	107754	62346

> 무작위 데이터를 사용하여 실행한 측정:

	<Insertion Sort>	<Quick Sort>
[1000]	483	88
[2000]	1990	136
[3000]	4355	208
[4000]	10600	273
[5000]	13752	352
[6000]	22468	427
[7000]	26830	693
[8000]	36257	667
[9000]	46977	744
[10000]	58286	797

<<< 정렬 성능 비교 프로그램을 종료합니다 >>>

□ 출력 자세히 보기: 오름차순 데이터

<<< 정렬 성능 비교 프로그램을 시작합니다 >>>

>> 2 가지 정렬의 성능 비교: 삽입, 퀵 <<

> 오름차순 데이터를 사용하여 실행한 측정:

	<Insertion Sort>	<Quick Sort>
[1000]	5	557
[2000]	10	2185
[3000]	13	4866
[4000]	19	8646
[5000]	27	17656
[6000]	33	19246
[7000]	35	26994
[8000]	42	37132
[9000]	42	51646
[10000]	49	57863

□ 출력 자세히 보기: 내림차순 데이터

> 내림차순 데이터를 사용하여 실행한 측정:

	<Insertion Sort>	<Quick Sort>
[1000]	968	593
[2000]	3654	2431
[3000]	11365	6779
[4000]	16745	11381
[5000]	26705	18949
[6000]	37453	26925
[7000]	49529	32725
[8000]	66185	42986
[9000]	77545	56056
[10000]	107754	62346

□ 출력 자세히 보기: 무작위 데이터

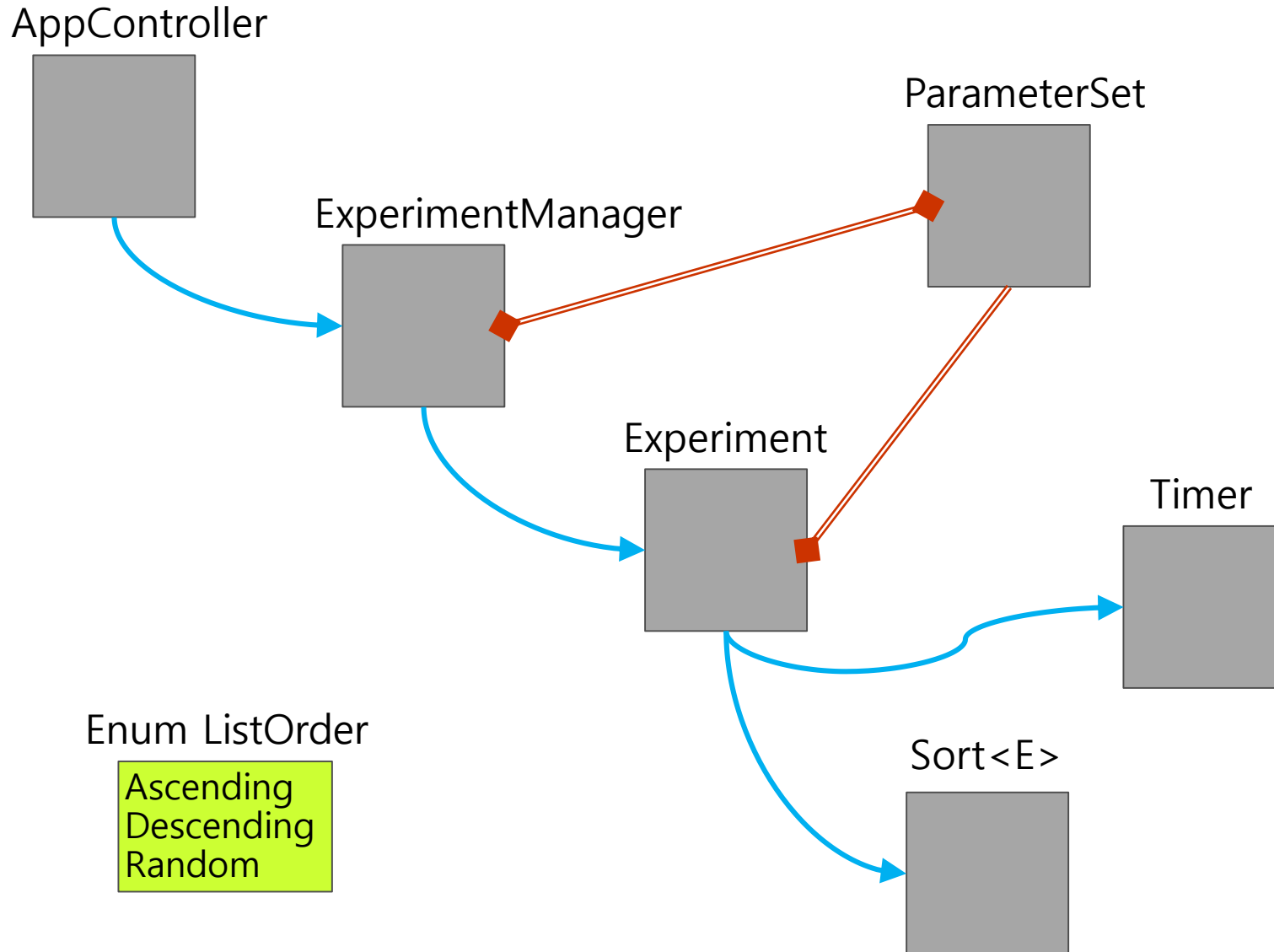
> 무작위 데이터를 사용하여 실행한 측정:

	<Insertion Sort>	<Quick Sort>
[1000]	483	88
[2000]	1990	136
[3000]	4355	208
[4000]	10600	273
[5000]	13752	352
[6000]	22468	427
[7000]	26830	693
[8000]	36257	667
[9000]	46977	744
[10000]	58286	797

<<< 정렬 성능 비교 프로그램을 종료합니다 >>>

실험의 구성

실험에 사용되는 객체들의 관계



□ 실험을 하는데 필요한 class [1]

■ Class "ExperimentManager"

- 측정 실험 전체를 관리한다.
- 주요 기능:
 - ◆ 실험의 매개변수 설정
 - ◆ 비교 분석할 정렬의 종류를 설정: 삽입정렬, 퀵정렬
 - ◆ 실험 데이터 생성
 - ◆ 실험 실시
 - ◆ 실험 결과 보관, 제공

■ Class "Experiment"

- 측정 실험을 실행한다
- 주요 기능:
 - ◆ ExperimentManager 객체가 제공한 매개변수 값을 사용하여, 측정 실험을 실시한다.

□ 실험을 하는데 필요한 class [1]

■ Class "ParameterSet"

- 실험에 필요한 매개변수 값들을 보관한다.
- 매개변수:
 - ◆ 실험에 사용되는 데이터의 시작 크기 (Starting Data Size)
 - ◆ 크기를 증가시키면서 실험을 반복하는 횟수 (Number Of Size-increasing Steps)
 - ◆ 반복마다 증가하는 데이터의 크기 (Increment Size)

■ Class "DataGenerator"

- 실험에 사용할 데이터를 생성한다
- 세 가지 데이터를 필요한 크기로 생성:
 - ◆ 오름차순 (ascending) 데이터
 - ◆ 내림차순 (descending) 데이터
 - ◆ 무작위 (random) 데이터

main()

main()을 위한 class

```
public class DS1_12_학번_이름 {  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        AppController appController= new AppController() ;  
        appController.run () ;  
    }  
}
```

Class "AppView"



□ AppView:

```
public class AppView
{
    // 비공개 인스턴스 변수
    private Scanner _scanner ;

    // 생성자
    public AppView () {
        this._scanner = new Scanner (System.in) ;
    }

    // 공개함수의 구현
    public void output (String aString) {
        ..... // 여기를 작성하시오
    }

    public void outputLine (String aString) {
        ..... // 여기를 작성하시오
    }
}
```



□ Class "AppView" 의 공개함수

■ 출력을 위한 공개 함수

- public void output (String aString)
 - ◆ 문자열 aString 을 출력
 - ◆ System.out.**print()** 를 사용
- public void outputLine (String aString)
 - ◆ 문자열 aString 을 출력
 - ◆ System.out.**println()** 을 사용

Enum “ListOrder”

Enum "ListOrder"

```
public enum ListOrder
{
    // 이번 실험에서는, 3 가지 유형의 데이터 리스트를 구분하고 있다.
    // 이 구분을 표현할 목적으로 Enum "ListOrder" 를 사용한다.
    Ascending,    // 오름차순 데이터 리스트의 유형을 표현
    Descending,   // 내림차순 데이터 리스트의 유형을 표현
    Random;       // 무작위 데이터 리스트의 유형을 표현

    // Enum 은 Class 의 특수한 경우로 간주된다.
    // 따라서 아래와 같이 상수를 선언할 수 있다.

    public static final String[] ORDER_NAMES = {"오름차순", "내림차순", "무작위"};

    // 또한 아래와 같이 member method 역시 선언할 수 있다.
    // 즉, Enum 안에 선언된 값들은 Enum 의 객체 인스턴스로 인식된다.
    // Ascending, Descending, Random 각각이 객체 인스턴스이다.

    public String orderName () {
        return ListOrder.ORDER_NAMES [ this.ordinal() ];
        // "ordinal()"은 모든 Enum 에 미리 정의되어 있는 함수로,
        // 선언된 값의 Enum 안에서의 순서를 정수로 얻을 수 있다.
        // 즉. Ascending.ordinal() 은 0, Descending.ordinal() 은 1,
        // Random.ordinal() 은 2 를 얻는다.
    }
}
```

□ ListOrder: 공개 인스턴스 함수 "orderName()"

■ public String orderName ()

- Enum 의 각 값에 문자열 이름을 부여하여, Enum 값에 해당하는 문자열을 얻으려는 것이다.

- 사용 예:

```
ListOrder order = ListOrder.Random ;  
this._appView.outputLine (order.orderName()) ;
```

- 미리 부여한 이름은 상수 배열 ORDER_NAMES[] 로 정의해 두고 있다:

```
public static String[] ORDER_NAMES = {"오름차순", "내림차순", "무작위"} ;
```

- ◆ this.ordinal() 값을 인덱스로 사용하여 ORDER_NAMES[] 배열에서 해당 문자열 이름을 얻을 수 있다.

Class “AppController”



□ ApplicationController: 개요 [1]

<<< 정렬 성능 비교 프로그램을 시작합니다. >>>			← 프로그램 시작 메시지
>> 2 가지 정렬의 성능 비교: 삽입, 퀵 <<			← 실험 내용 설명 메시지
> 오름차순 데이터를 사용하여 실행한 측정:			
	<Insertion Sort>	<Quick Sort>	
[1000]	5	557	
[2000]	10	2185	
[3000]	13	4866	
[4000]	19	8646	
[5000]	27	17656	← 오름차순 데이터를 사용하여 얻은 측정 결과: measureAndShow (ListOrder.Ascending)
[6000]	33	19246	
[7000]	35	26994	
[8000]	42	37132	
[9000]	42	51646	
[10000]	49	57863	
> 내림차순 데이터를 사용하여 실행한 측정:			
	<Insertion Sort>	<Quick Sort>	
[1000]	968	593	
[2000]	3654	2431	
[3000]	11365	6779	
[4000]	16745	11381	
[5000]	26705	18949	← 내림차순 데이터를 사용하여 얻은 측정 결과: measureAndShow (ListOrder.Descending)
[6000]	37453	26925	
[7000]	49529	32725	
[8000]	66185	42986	
[9000]	77545	56056	
[10000]	107754	62346	
> 무작위 데이터를 사용하여 실행한 측정:			
	<Insertion Sort>	<Quick Sort>	
[1000]	483	88	
[2000]	1990	136	
[3000]	4355	208	
[4000]	10600	273	
[5000]	13752	352	← 무작위 데이터를 사용하여 얻은 측정 결과: measureAndShow (ListOrder.Random)
[6000]	22468	427	
[7000]	26830	693	
[8000]	36257	667	
[9000]	46977	744	
[10000]	58286	797	
<<< 정렬 성능 비교 프로그램을 종료합니다. >>>			← 프로그램 종료 메시지

□ ApplicationController: 개요 [2]

- 3 가지 유형의 데이터 별로, 각각 성능 측정을 하여 결과를 보여준다.
- 하나의 데이터 유형마다, 테이블 형태로 다음의 3 가지를 출력한다:

> 내림차순 데이터를 사용하여 실행한 측정:

<Insertion Sort>

<Quick Sort>

[1000]	968	593
[2000]	3654	2431
[3000]	11365	6779
[4000]	16745	11381
[5000]	26705	18949
[6000]	37453	26925
[7000]	49529	32725
[8000]	66185	42986
[9000]	77545	56056
[10000]	107754	62346

← 테이블 제목: showTableTitle()

← 테이블 헤드: showTableHead()

← 측정 결과 : showMeasuredResult()

□ ApplicationController: 인스턴스 변수, 생성자

```
public class ApplicationController {  
  
    // 비공개 변수들  
    private AppView      _appView ;  
    private ExperimentManager _manager ;  
  
    // 생성자  
    public ApplicationController() {  
        this._appView = new AppView() ;  
        this._manager = new ExperimentManager() ;  
    }  
  
    // 비공개함수의 구현  
    .....  
  
    // 공개함수의 구현  
    .....  
} // End of class "AppController"
```

□ ApplicationController: showTableTitle(), showTableHead(), showMeasuredResult()

```
private void showTableTitle (ListOrder anOrder) {
    this._appView.outputLine (
        "> " + anOrder.orderName() + " 데이터를 사용하여 실행한 측정:" );
}

private void showTableHead () {
    this._appView.outputLine (
        String.format ("%8s", " ") +
        String.format ("%16s", "<Insertion Sort>") +
        String.format ("%16s", "<Quick Sort>")
    );
}

private void showMeasuredResult () {
    int startingSize = this._manager.parameterSet().startingSize();
    int incrementSize = this._manager.parameterSet().incrementSize();
    int numberOfSteps = this._manager.parameterSet().numberOfSizeIncreasingSteps();

    for ( int step = 0 ; step < numberOfSteps ; step++ ) {
        int sortingSize = startingSize + (incrementSize * step);
        this._appView.outputLine (
            "[" + String.format ("%5d", sortingSize) + "]" +
            String.format ("%16d", this._manager.measuredResultForInsertionSortAt (step)) +
            String.format ("%16d", this._manager.measuredResultForQuickSortAt (step))
        );
    }
}
```

□ ApplicationController: run(), measureAndShow()

```
private void measureAndShowResultFor (ListOrder anOrder) {
    // 주어진 anOrder 에 대하여, 성능을 측정하여 결과를 보여준다.
    this.showTableTitle (anOrder) ;
    this.showTableHead () ;
    this._manager.performExperiment (anOrder);
    this.showMeasuredResult () ;
    this._appView.outputLine ("") ;
}

public void run () {
    this._appView.outputLine ("<<<정렬 성능 비교 프로그램을 시작합니다 >>>") ;
    this._appView.outputLine ("") ;
    {
        this._appView.outputLine (">> 2 가지 정렬의 성능 비교: 삽입, 퀵 <<") ;
        this._manager.prepareExperiment (null) ;
        // ExperimentManager 객체에게 실험을 준비시킨다.
        // 이번 실험에서는 매개변수 값으로 기본 설정 값을 사용한다.
        // 기본 설정 값은 Class "ExperimentManager" 에 설정되어 있다.

        this.measureAndShowResultFor (ListOrder.Ascending) ;
        this.measureAndShowResultFor (ListOrder.Descending) ;
        this.measureAndShowResultFor (ListOrder.Random) ;
    }
    this._appView.outputLine ("<<< 정렬 성능 비교 프로그램을 종료합니다 >>>") ;
}
```

Class “DataGenerator”

□ "Static" Class

■ Static Class:

- 객체 인스턴스를 필요로 하지 않는 class. 즉 객체를 생성할 필요가 없는 class.
- 예: Math
- 오로지 static constant, static variable, 그리고 static method 만 존재하게 된다.

■ Java 에서는 "Static" class 를 선언하는 방법을 직접적으로 제공하지는 않는다. 즉 다음과 같이 선언할 수 없다.

- ```
public static class Math { // Compile Error!!

}
```

## ■ 그러나, 실질적인 static class 를 만들 수는 있다:



# Java 에서 static class 선언 방법

- class 의 선언은 "final":
  - public **final** class **DataGenerator** {
    - .....
    - }
  - 일반적으로 "Math" 와 같은 라이브러리를 만들 때 사용하게 된다. 라이브러리를 상속받는다?
  - 그럼에도, Static class 자체를 확장하고 싶다면?
    - ◆ static variable 이나 static method 를 추가한 확장된 static class 를 만들고 싶다면?
- 생성자는 "private" 으로 선언:
  - **private** DataGanarator() { }
  - 객체 인스턴스를 생성할 수 없게 막아 놓는다.
- 모든 상수, 변수, 함수는 "static" 으로 선언:
  - public **static** Integer[] **ascendingOrderList** (int aSize) {
    - .....
    - }
  - 사용법:
    - Integer[] myAscendingList = **DataGenerator.ascendingOrderList** (100) ;

# □ Static class "DataGenerator" [1]

```
public final class DataGenerator {
 // Static class.
 // 더 이상 상속할 필요가 없으므로 "final" 로 선언

 // 생성자는 private. 따라서 외부에서 객체를 만들 수 없다.
 private DataGenerator() {} ;

 // 모든 공개함수는 static
 public static Integer[] ascendingOrderList (int aSize) {
 Integer[] list = null ;
 if (aSize > 0) {
 list = new Integer[aSize] ;
 for (int i = 0 ; i < aSize ; i++) {
 list[i] = i ;
 }
 }
 return list ;
 }

 public static Integer[] descendingOrderList (int aSize) {
 // 여기를 작성하시오
 }
}
```

# □ Static class "DataGenerator" [2]

```
public final class DataGenerator {

 public static Integer[] randomOrderList (int aSize) {
 // 겹치는 원소가 없는 무작위순 리스트를 생성하여, 돌려준다.
 Integer[] list = null ;
 if (aSize > 0) {
 // 일단 Ascending order list 를 만든다
 list = new Integer[aSize] ;
 for (int i = 0 ; i < aSize ; i++) {
 list[i] = i ;
 }

 // 각 원소 list[i] 에 대해 무작위 위치 j 를 생성하여 list[i] 와 list[j] 를 맞바꾼다.
 Random random = new Random() ;
 for (int i = 0 ; i < aSize ; i++) {
 int j = random.nextInt (aSize) ;
 Integer temp = list[i] ;
 list[i] = list[j] ;
 list[j] = temp ;
 }
 }
 return list ;
 }
}
```

# Class "ParameterSet"



# □ ParameterSet: 개요

- 단순히 실험에서 필요한 매개변수들을 모아 놓는 역할을 한다.
- 사용하는 매개변수:
  - `private int _startingSize ;`
  - `private int _numberOfSizeIncreasingSteps ;`
  - `private int _incrementSize ;`
- 생성자:
  - 주어진 값으로 매개변수들을 초기화 할 수 있도록 한다.
- 각 인스턴스 변수의 `getter/setter` 를 `public` 으로!
- 실험 데이터 리스트의 최대 크기를 얻을 수 있는 함수 "`maxDataSize()`" 를 만든다.

# □ ParameterSet: 구현

```
public class ParameterSet {

 // Private instance variables
 private int _startingSize ;
 private int _numberOfSizeIncreasingSteps ;
 private int _incrementSize ;

 // public Getters & Setters
 // 여기를 작성하시오: 모든 인스턴스 변수의 getter 와 setter를 만든다.

 public int maxDataSize () {
 return (this.startingSize() +
 (this.incrementSize() * (this.numberOfSizeIncreasingSteps() - 1))
) ;
 }

 // 생성자:
 public ParameterSet (int givenStartingSize,
 int givenNumberOfSizeIncreasingSteps,
 int givenIncrementSize)
 {
 // 여기를 작성하시오
 }
}
```

# Class “Timer”



# □ Timer: 개요

- 실행 시간을 측정할 수 있는 타이머를 정의한다
  - Java 의 System.nanoTime() 을 사용한다. :
  - 단위: 나노 초 ( $10^{-9}$  second)
  - 더 이상 상속될 필요가 없으므로, "final" class 로 선언한다.

```
public final class Timer {
 // private instance variables
 private long _startTime ;
 private long _stopTime ;

 // Constructor
 public Timer () { } ;

 // public methods
 public void start () {
 this._startTime = System.nanoTime () ;
 }

 public void stop () {
 this._stopTime = System.nanoTime () ;
 }

 public long duration () {
 return (this._stopTime - this._startTime) ;
 }
}
```

# Class “Experiment”



# □ Experiment: 개요

- 정렬 성능 측정 실험을 실행한다.
- 실험에 미리 주어지는 것들:
  - 매개변수 집합 / 정렬할 데이터 / 정렬 방법
- 측정
  - `private Integer[] copyListOfGivenSize (Integer[] aList, int copiedSize)`
    - ◆ 주어진 `aList[]` 에서 `copiedSize` 만큼을 복사한 리스트를 얻는다.
    - ◆ `aList[0]` 부터 `aList[copiedSize-1]` 까지 복사한다.
  - `private long durationOfSingleSort (Sort<Integer> aSort, Integer[] aList)`
    - ◆ 주어진 `aList[]` 를 사용하여 정렬 (`aSort`) 을 실행하여, 실행 시간을 측정한다.
  - `public long[] durationsOfSort (Sort<Integer> aSort, Integer[] experimentList)`
    - ◆ 정렬 방법 (`aSort`) 과, 정렬의 성능 측정에 사용할 리스트 `experimentList[]` 가 주어진 다.
    - ◆ 매개변수 값을 사용하여 데이터 크기 별로 정렬 시간을 측정하여, 그 결과를 배열로 얻는다.

# □ Experiment: 인스턴스 변수, 생성자

```
public class Experiment {
```

```
// Private instance variables
```

```
private final ParameterSet _parameterSet ;
```

```
// Class의 함수에서 이 값을 변경할 수 없다.
```

```
// 오로지, 생성자 안에서만 값을 설정할 수 있다.
```

```
// 즉, 객체를 생성할 때 정해진 값을 그대로 유지한다.
```

```
// 생성자:
```

```
public Experiment (ParameterSet givenParameterSet) {
 this._parameterSet = givenParameterSet ;
}
```

# ❏ Experiment: private methods

```
public class Experiment {

 // Private methods
 private Integer[] copyListOfGivenSize (Integer[] aList, int copiedSize) {
 Integer[] copiedList = null ;
 if (copiedSize <= aList.length) {
 copiedList = new Integer [copiedSize] ;
 for (int i = 0 ; i < copiedSize ; i++) {
 copiedList[i] = aList[i] ;
 }
 }
 return copiedList ;
 }

 private long durationOfSingleSort (Sort<Integer> aSort, Integer[] aList) {
 Timer timer = new Timer() ;
 timer.start() ;
 {
 aSort.sort (aList) ;
 }
 timer.stop() ;
 return timer.duration() ;
 }
}
```

# □ Experiment: public method

```

public long[] durationsOfSort (Sort<Integer> aSort, Integer[] experimentList) {
 // 정렬 방법이 매개변수로 주어져 있음에 유의할 것: Class "Sort" 를 볼 것

 int numberOfSteps = this._parameterSet.numberOfSizeIncreasingSteps() ;
 // 크기 별로 실행할 측정 횟수
 long[] durations = new long[numberOfSteps] ; // 측정 결과를 저장할 곳

 int sortingSize = this._parameterSet.startingSize() ; // 정렬 데이터의 시작 크기
 int incrementSize = this._parameterSet.incrementSize() ; // 정렬 데이터의 증가 크기

 for (int step = 0 ; step < numberOfSteps ; step++) {
 Integer[] listForSorting = this.copyListOfGivenSize (experimentList, sortingSize) ;
 // 측정에 사용할 데이터 리스트 복사
 durations[step] = this.durationOfSingleSort (aSort, listForSorting) ;
 // 측정하여, 그 결과를 저장
 sortingSize += incrementSize ; // 다음 단계의 정렬 데이터 크기를 얻는다
 }
 return durations ;
}

} // End of class "Experiment"

```

# Class “ExperimentManager”



## ❑ ExperimentManager: 상수

```
public class ExperimentManager {
 // Constants for the parameters:
 private static final int DEFAULT_NUMBER_OF_SIZE_INCREASING_STEPS = 10 ;
 private static final int DEFAULT_INCREMENT_SIZE = 1000 ;
 private static final int DEFAULT_STARTING_SIZE = DEFAULT_INCREMENT_SIZE ;

 // Constants: 실험에서 사용할 정렬들을 상수 객체로 선언
 private static final InsertionSort<Integer>
 INSERTION_SORT = new InsertionSort<Integer>() ;
 private static final QuickSort<Integer>
 QUICK_SORT = new QuickSort<Integer>() ;
}
```

## ❑ ExperimentManager: 인스턴스 변수, 생성자

```
public class ExperimentManager {
 // Constants for the parameters:

 // Constants: 실험에서 사용할 정렬들을 상수 객체로 선언

 // Private instance variables
 private Experiment _experiment ; // 측정 실험을 실시할 객체
 private ParameterSet _parameterSet ; // 측정 실험에 사용할 매개변수 집합
 private Integer[] _ascendingList ; // 측정에서 정렬에 사용할 오름차순 데이터 리스트
 private Integer[] _descendingList ; // 측정에서 정렬에 사용할 내림차순 데이터 리스트
 private Integer[] _randomList ; // 측정에서 정렬에 사용할 무작위 데이터 리스트
 private long[] _measuredResultForInsertionSort ; // 삽입 정렬의 측정 결과 저장할 곳
 private long[] _measuredResultForQuickSort ; // 퀵 정렬의 측정 결과 저장할 곳

 // Public Getter
 public ParameterSet parameterSet () {
 return this._parameterSet ;
 }

 // Constructor
 public ExperimentManager () {
 this.setParameterSetWithDefaults() ; // 기본 값으로 매개변수 집합을 초기화한다
 }
}
```

# ❑ ExperimentManager: Private methods [1]

```
public class ExperimentManager {

 // private methods
 private void prepareExperimentLists () {
 int maxSize = this._parameterSet.maxDataSize() ;

 this._ascendingList = DataGenerator.ascendingOrderList (maxDataSize) ;
 this._descendingList = DataGenerator.descendingOrderList (maxDataSize) ;
 this._randomList = DataGenerator.randomOrderList (maxDataSize) ;
 }

 private void setParameterSetWithDefaults () {
 this._parameterSet = new ParameterSet (
 DEFAULT_STARTING_SIZE,
 DEFAULT_NUMBER_OF_SIZE_INCREASING_STEPS,
 DEFAULT_INCREMENT_SIZE) ;
 }
}
```

## ❏ ExperimentManager: Private methods [2]

```
private Integer[] experimentListOfOrder (ListOrder anOrder)
{
 // 주어진 anOrder 에 해당하는 리스트를 돌려준다.

 switch (anOrder) {
 case Ascending:
 return this._ascendingList ;
 case Descending:
 return this._descendingList ;
 default:
 return this._randomList;
 }
}
```

## ❏ ExperimentManager: Public methods [1]

```
public void prepareExperiment (ParameterSet aParameterSet) {
 // 실험을 준비한다.

 if (aParameterSet != null) {
 // 객체 생성할 때, 매개변수 집합은 기본 값으로 설정되어 있다.
 // 실험 준비 단계에서, 이렇게 새로운 매개변수 집합을 주어 변경할 수 있다.
 this._parameterSet = aParameterSet ;
 }

 this._experiment = new Experiment (this._parameterSet) ;
 // 현재 상태의 매개변수 집합을 사용하여 Experiment 객체를 생성한다.

 this.prepareExperimentLists () ;
 // 측정 실험에서 정렬에 사용할 데이터 리스트를 생성하여 보관한다.

 // 다음의 내용은 생략 가능한, 내용적으로는 의미가 없는 실행이다.
 // 단지 실험 측정 결과를 안정화시키기 위한 목적일 뿐이다.
 this.performExperiment (ListOrder.Random) ;
 this.performExperiment (ListOrder.Random) ;
}
```

## ❏ ExperimentManager: Public methods [2]

```
public long measuredResultForInsertionSortAt (int sizeStep) {
 return this._measuredResultForInsertionSort [sizeStep] ;
}

public long measuredResultForQuickSortAt (int sizeStep) {
 return this._measuredResultForQuickSort [sizeStep] ;
}

public void performExperiment (ListOrder anOrder) {
 // 측정 실험을 실행한다.

 // 주어진 anOrder 의 실험 리스트를 얻는다.
 Integer[] experimentList = this.experimentListOfOrder (anOrder) ;

 // 이 실험 리스트로 삽입과 퀵 각각의 정렬의 성능을 측정해서, 그 결과를 얻는다.
 this._measuredResultForInsertionSort =
 this._experiment.durationsOfSort (INSERTION_SORT, experimentList) ;
 this._measuredResultForQuickSort =
 this._experiment.durationsOfSort (QUICK_SORT, experimentList) ;
}
```

# Abstract Class "Sort<E>"



# □ Abstract Class:

- Abstract (추상) class 로는 객체를 생성할 수 없다.
  - 반드시 상속받는 class 가 있어서, 그 상속받은 class를 사용하여 객체를 생성할 수 있다.
- 인스턴스 변수:
  - "private" 으로 선언하면 상속이 되지 않는다.
  - 상속받는 class 에서 사용하려면 반드시 "protected" 로 선언되어야 한다.
  - 인스턴스 변수이므로 "public" 으로 선언하는 것은 피해야 할 것이다.
- 생성자:
  - "protected" 로 선언한다.
- 멤버 함수:
  - Abstract class 자체에서 이미 구현되는 함수:
    - ◆ 상속받는 class 에서 사용하려면 반드시 "protected" 또는 "public" 으로 선언되어야 한다.
  - Abstract class 자체에서는 구현되지 않고, 상속받는 class 에서 override 하여 구현할 함수:
    - ◆ Abstract method 로 선언해야 한다.

# □ Abstract Class:

- InsertionSort 와 QuickSort 는 정렬 구현 방법은 다르다.
- 그러나, 사용자의 관점에서 사용법은 동일할 수 있다.
- 사용법과 구현 방법이 동일한 멤버 함수는 공통으로 만들어 사용하려면?
  - swap() 과 같은 일부 함수는 사용법은 물론, 구현 방법도 동일하다. 이러한 기능을 각각의 정렬 마다 구현하는 것은 낭비적이다.
- 두 정렬의 동일한 사용법을 뽑아서 하나의 abstract class "sort" 로 정의:
  - 구현자의 관점:
    - ◆ InsertionSort 와 QuickSort 각각의 정렬은 이 abstract class "Sort" 를 상속받도록 하는 것이 효과적인 코드 관리 방법.
    - ◆ 예:
 

```
public class InsertionSort<E> extends Sort<E> {...}
```
  - 사용자의 관점:
    - ◆ 객체 변수의 선언은 abstract class "Sort" 로 하고, 객체의 생성만 필요에 맞추어 특정 정렬 방법의 객체로 생성한다, 나중에 InsertionSort 를 QuickSort 로 바꾸고 싶으면, 객체 생성만 QuickSort 객체로 생성하도록 바꾸면 되며, 다른 부분은 수정할 필요가 없다.
    - ◆ 예:
 

```
Sort<Integer> mySort = InsertionSort<Integer>();
mySort.sort (list) ;
```

# □ Abstract class "Sort" : 구현

```
public abstract class Sort<E extends Comparable<E> >
```

```
{
```

```
// Protected Method
```

```
protected void swap (E[] aList, int i, int j) {
```

```
 E temp = aList[i];
```

```
 aList[i] = aList[j];
```

```
 aList[j] = temp;
```

```
}
```

```
// Constructor
```

```
protected Sort () {
```

```
}
```

```
// Public Method
```

```
public abstract boolean sort (E[] aList, int aSize);
```

```
} // End of "Sort"
```

- 정렬 리스트의 원소의 자료형은 고정시키지 않는 것이 유연한 구현 방법이므로, 원소의 자료형으로 generic type E 를 선언하고 있다.
- "E extends Comparable<E>" 은, 정렬에서 원소끼리 비교할 수 있어야 하므로, E 로 주어지는 실제 원소의 class 는 반드시 interface "Comparable" 을 "implements" 하고 있어야 함을 요구하는 선언이다. 따라서 E 의 실제 class 는 함수 "compareTo()" 를 override 하여 구현하고 있어야 함을 요구하고 있다.
- 이 과제에서는, E 는 class "Integer"로 주어지게 된다. Java 에서 제공하고 있는 "Integer" 는 이미 "Comparable" 하며, "compareTo()" 역시 이미 구현되어 있다. 그러므로 이 과제에서는 "compareTo()" 를 구현할 필요는 없다.

# Class “InsertionSort”



# □ InsertionSort: 개요

## ■ 맨 앞 원소의 값을 $(-\infty)$ 로:

- 이렇게 하면, 원소 삽입 위치를 찾는 while loop 조건이 비교를 한 번만 하게 되어 시간적으로 유리하다.
- 구현 방법:
  - ◆ 가장 작은 원소를 찾아 맨 앞에 갖다 놓기로 한다.
  - ◆ 이 원소는 실질적인  $-\infty$  값의 역할을 한다.

## ■ Abstract class "Sort"를 상속받는다:

- "swap()" 은 상속받은 것을 그대로 사용한다.
- Abstract method 인 sort() 는 override 하여 구현한다.

# □ InsertionSort: 구현

```

public class InsertionSort<E extends Comparable<E>> extends Sort<E> {
 // Constructor
 public InsertionSort () { }

 // Public Method
 public boolean sort (E[] aList, int aSize) {
 if ((aSize < 1) || (aSize > aList.length)) {
 return false ;
 }
 int minLoc = 0 ;
 for (int i = 1 ; i < aSize ; i++) {
 if (aList[i].compareTo(aList[minLoc]) < 0) {
 minLoc = i ;
 }
 }
 this.swap (aList, 0, minLoc) ; // Abstract class "Sort" 에 구현된 것을 그대로 사용하고 있다.

 for (int i = 2 ; i < aSize ; i++) {
 E insertedElement = aList[i] ;
 int insertionLoc = i - 1 ;
 while (aList[insertionLoc].compareTo(insertedElement) > 0) {
 aList[insertionLoc+1] = aList[insertionLoc] ;
 }
 // While loop 조건이 false 라서 loop 종료. 따라서, (insertionLoc+1) 이 원소 삽입 위치.
 aList[insertionLoc+1] = insertedElement ;
 }
 return true;
 }
} // End of "Sort"

```

# Class “QuickSort”



# □ QuickSort: 개요

## ■ 맨 뒤 원소의 값을 $(+\infty)$ 로:

- 퀵 정렬에서는 맨 뒤에  $+\infty$  가 존재해야 한다.
  - ◆ 그렇지 않다면?
- 구현 방법:
  - ◆ 정렬 구간:  $[0, \text{aSize}-1]$
  - ◆ 가장 큰 원소를 찾아 맨 뒤인  $(\text{aSize}-1)$  위치에 갖다 놓는다.
  - ◆ 이 원소는 실질적인  $+\infty$  값의 역할을 한다.
  - ◆ 실제 퀵 정렬 구간은  $[0, \text{aSize}-2]$  가 된다.

## ■ Abstract class "Sort"를 상속받는다:

- swap() 은 상속받은 것을 그대로 사용한다.
- Abstract method 인 sort() 는 override 하여 구현한다.

# □ QuickSort: 필요한 함수들

- private int **pivot** (E[] aList, int left, int right) ;
  - 주어진 구간에서 pivot 원소 위치를 결정하여 얻는다.
  - 현재는, 구간의 가장 왼쪽 원소 위치를 사용하기로 한다.
  
- private int **partition** (E[] aList, int left, int right) ;
  - 주어진 구간을 partition 한다.
  - Partition 을 실행한 후에, pivot 원소가 최종적으로 놓인 위치를 얻는다.
  
- private void **quickSortRecursively** (E[] aList, int left, int right) ;
  - 주어진 구간을 재귀적으로 퀵 정렬 한다.
  
- public boolean **sort** (E[] aList, int aSize) ;
  - Abstract class Sort<E> 의 sort()를 override 하여 구현한다.

# □ QuickSort: 구현 [1]

```
public class QuickSort<E extends Comparable<E>> extends Sort<E> {
 // Constructor
 public QuickSort () { }

 // Private methods
 private int pivot (E[] aList, int left, int right) {
 return left ;
 }
 private int partition (E[] aList, int left, int right) {
 // 여기를 작성하시오
 }
 private void quickSortRecursively (E[] aList, int left, int right) {
 // 여기를 작성하시오
 }
}
```

# □ QuickSort: 구현 [2]

```

public class QuickSort<E extends Comparable<E>> extends Sort<E> {

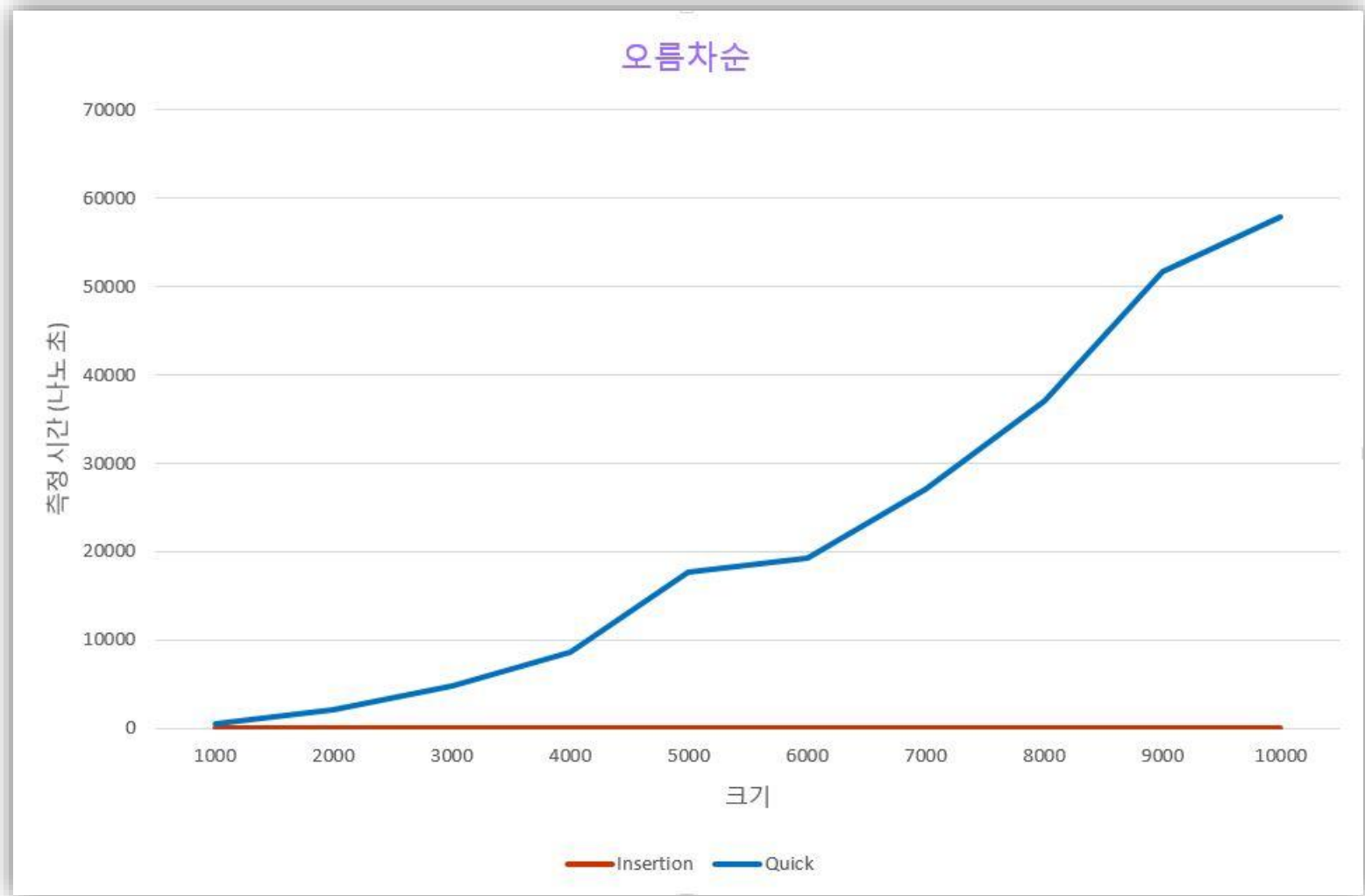
 @Override
 public boolean sort (E[] aList, int aSize) {
 if (aSize < 1 || aSize > aList.length) {
 return false ;
 }
 int maxLoc = 0 ;
 for (int i = 1 ; i < aSize ; i++) {
 if (aList[i].compareTo(aList[maxLoc] > 0) {
 maxLoc = i ;
 }
 }
 this.swap (aList, maxLoc, aSize-1) ;

 this.quickSortRecursively (aList, 0 , aSize-2) ;
 return true ;
 }
} // End of "Sort"

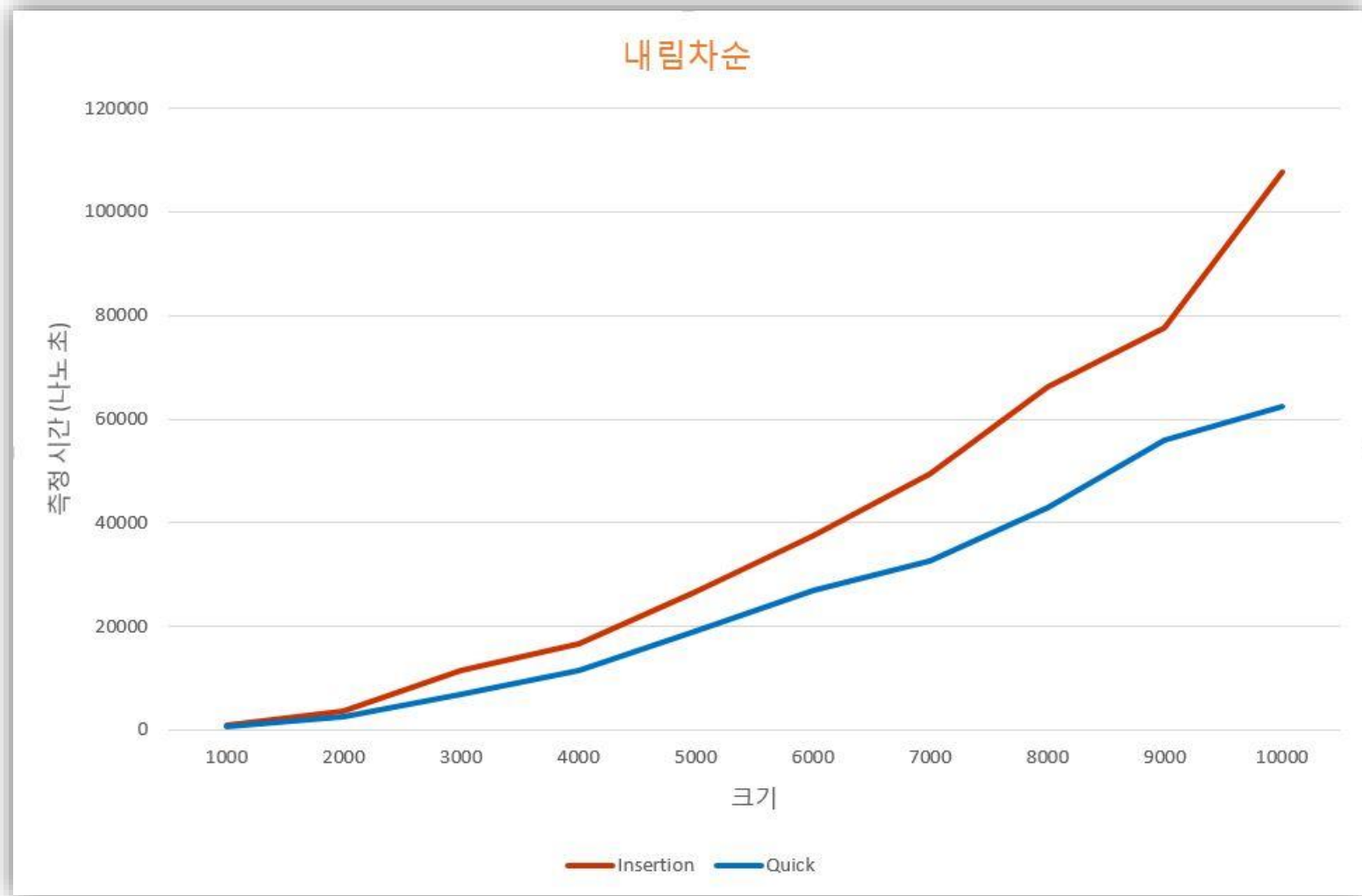
```

# 요약

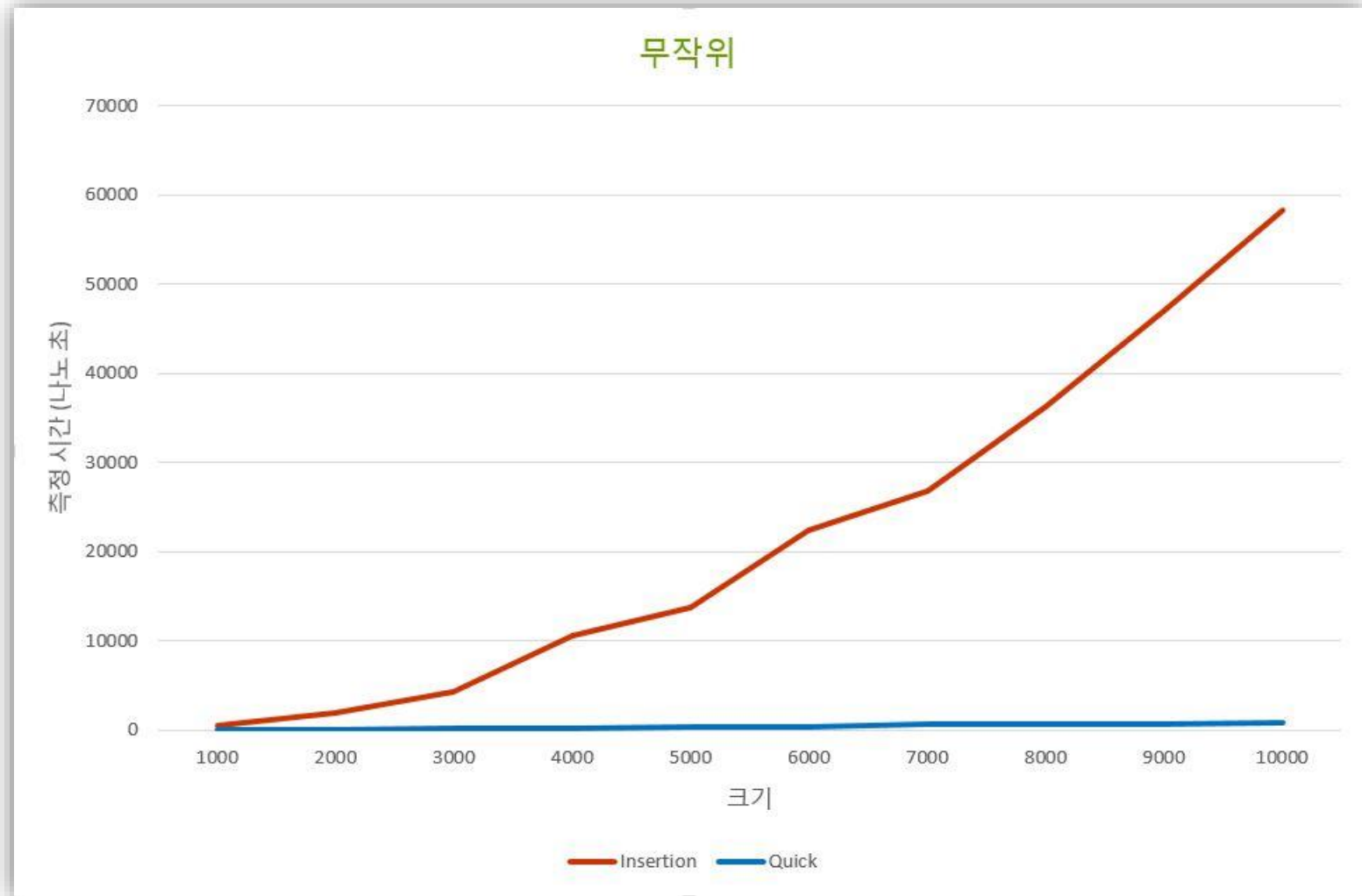
# □ 측정 결과를 그래프로: 오름차순



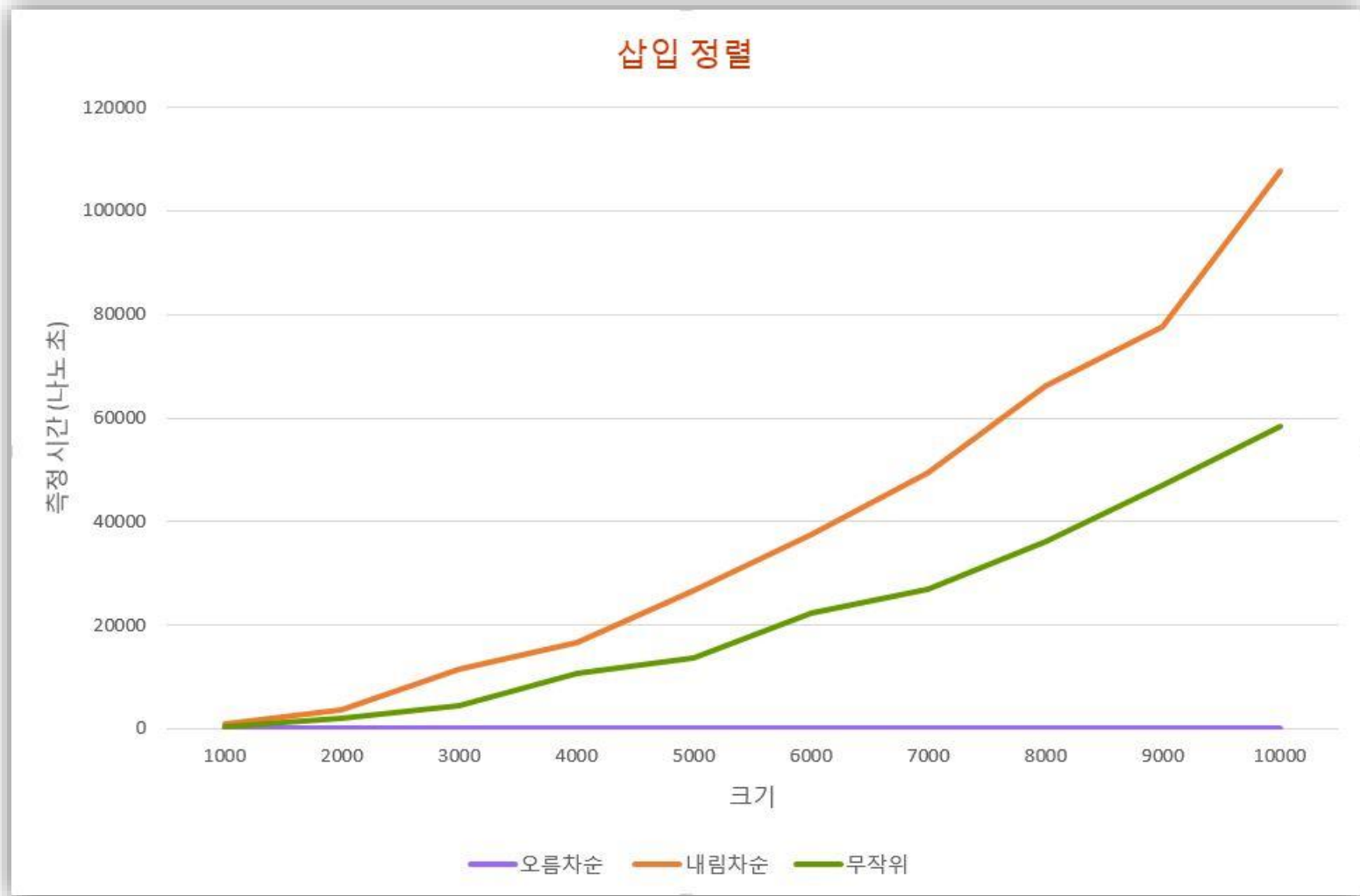
# □ 측정 결과를 그래프로: 내림차순



# □ 측정 결과를 그래프로: 무작위



# □ 측정 결과를 그래프로: 삽입 정렬



# □ 측정 결과를 그래프로: 퀵 정렬



## □ 성능측정 결과분석

- 각 정렬의 성능을 측정한 자신의 결과를 사용하여, 엑셀을 이용하여 그래프를 그려서, 보고서에 첨부한다.
- 자신의 그래프를 보고, 각 경우에 있어서 이론적 시간 복잡도와 비교 분석하여, 그 내용을 보고서에 작성한다.

# 보고서 작성과 제출



# □ PDF 보고서 작성 방법

## ■ 겉장

- 제목: 자료구조 실습 보고서
- 부제목: [제12주] 정렬 성능 비교
- 제출일
- 학번/이름

## ■ 내용

1. 프로그램 설명서
  1. 주요 알고리즘 /자료구조 /기타
  2. 함수 설명서
  3. 종합 설명서
2. 프로그램 장단점 분석
3. 실행 결과 분석
  1. 입력과 출력 (화면 capture 시킬 것)
  2. 결과 분석
4. 소스코드

# □ 과제 제출

## ■ 파일 이름 작명 방법

● DS1\_12\_학번\_이름.zip

● 폴더의 구성

◆ DS1\_12\_학번\_이름

■ 프로그램

– 프로젝트 폴더 / 소스

– 메인 클래스 이름 : DS1\_12\_학번\_이름.java

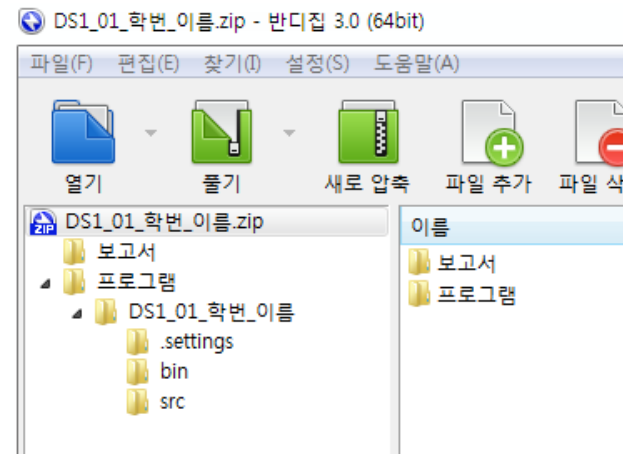
■ 보고서

– 이곳에 보고서 문서 파일을 저장한다.

– 입력과 실행 결과는 화면 image로 문서에 포함시킨다.

– 문서는 pdf 파일로 만들어 제출한다.

– 실제 테스트를 진행 한 데이터(엑셀 파일)를 제출한다.



# [제 12 주] 끝



